

WorkoutLang

Design Specification Document — D1 (Part 2)

CSE 341 — Concepts of Programming Languages

Gebze Technical University | Spring 2026

Implementation Language: C

4.1 Language Overview

WorkoutLang is a domain-specific language for designing, structuring, and documenting resistance-training and cardio workout programs. Its target users are fitness coaches and athletes who want to describe workout sessions — exercises, sets, repetitions, rest periods, and progression rules — in a readable, unambiguous format without learning a general-purpose programming language.

Evaluation Against Sebesta's Language Evaluation Criteria (Ch. 1):

Readability (§1.3.1): WorkoutLang prioritizes readability above all. Keywords such as set, reps, rest, load, and repeat directly mirror the spoken vocabulary of fitness coaching. A trainer can read a WorkoutLang program aloud and understand it without formal language training.

Writability (§1.3.2): The language is narrow by design, offering only the constructs needed for workout programming. This intentional restriction improves writability for the target domain while sacrificing generality.

Reliability (§1.3.3): Strong typing prevents category errors such as using a duration value where a repetition count is expected. The type system is knowingly prioritized over flexibility.

Cost: WorkoutLang knowingly sacrifices computational generality and Turing-completeness. It is not a general-purpose language; its goal is expressive power within the fitness domain at minimal learning cost.

4.2 Lexical Structure

The following table lists all token categories recognized by the WorkoutLang lexer, along with their regular expressions or enumerated values.

Token Category	Pattern / Enumerated Values	Examples
IDENT	[a-zA-Z_][a-zA-Z0-9_]* (not a keyword)	benchpress, myWeek, vol
INT_LIT	[0-9]+	3, 10, 90
FLOAT_LIT	[0-9]+ "." [0-9]+	1.5, 72.5
DURATION_LIT	[0-9]+ ("s" "min" "hr")	90s, 2min, 1hr
WEIGHT_LIT	[0-9]+ ("kg" "lb")	80kg, 135lb
STRING_LIT	"\" [^\n]* \""	"Deload next week"
BOOL_LIT	true false	true, false
KEYWORDS	workout routine day set reps rest repeat weeks load if else while return print int float bool duration weight func	set, reps, if

OPERATORS	+ - * / == != < <= > >= && !	>, ==, &&
SEPARATORS	{ } () , : x	{, }, x

4.3 Syntax (EBNF Grammar)

The complete grammar is given in Sebesta EBNF notation (§3.1–3.2). Terminal symbols appear in double quotes or ALL_CAPS. Curly braces { } denote zero or more repetitions; square brackets [] denote optional elements. Operator precedence is encoded directly in the grammar hierarchy.

Top-Level Structure

```
<program> ::= { <top_decl> }
<top_decl> ::= <workout_decl> | <routine_decl> | <func_decl>
```

Note: variable declarations (<var_decl>) are not permitted at top level. They may only appear inside routine, function, or block bodies. This is consistent with §4.6 which defines three scope levels — global (workout/routine/func names), routine/function scope, and block scope — none of which covers a free-standing top-level variable. The grammar enforces this restriction syntactically.

Workout Declaration (domain-specific construct)

```
<workout_decl> ::= "workout" IDENT "{" { <day_decl> } "}"
<day_decl>      ::= "day" IDENT "{" { <set_stmt> } "}"
<set_stmt>      ::= "set" IDENT "reps" INT_LIT "x" INT_LIT "rest" DURATION_LIT
```

Routine & Function Declaration

```
<routine_decl> ::= "routine" IDENT "{" { <stmt> } "}"
<func_decl>    ::= "func" IDENT "(" [ <param_list> ] ")" ":" <type> <block>
<param_list>   ::= <param> { ", " <param> }
<param>        ::= IDENT ":" <type>
```

Types

```
<type> ::= "int" | "float" | "bool" | "duration" | "weight"
```

Statements

```
<block>        ::= "{" { <stmt> } "}"
<stmt>         ::= <var_decl> | <assign_stmt> | <if_stmt> | <while_stmt>
                  | <repeat_stmt> | <load_stmt> | <print_stmt>
                  | <return_stmt> | <set_stmt>
<var_decl>     ::= <type> IDENT "=" <expr>
<assign_stmt>  ::= IDENT "=" <expr>
<if_stmt>      ::= "if" <expr> <block> [ "else" <block> ]
<while_stmt>   ::= "while" <expr> <block>
<repeat_stmt>  ::= "repeat" INT_LIT "weeks" <block>
<load_stmt>    ::= "load" IDENT
<print_stmt>   ::= "print" <expr>
<return_stmt>  ::= "return" <expr>
```

Expressions (precedence encoded top-to-bottom, lowest to highest)

```
<expr>      ::= <or_expr>
<or_expr>    ::= <and_expr> { "|" <and_expr> }
<and_expr>   ::= <eq_expr> { "&&" <eq_expr> }
<eq_expr>    ::= <rel_expr> { ( "==" | "!=" ) <rel_expr> }
<rel_expr>   ::= <add_expr> { ( "<" | "<=" | ">" | ">=" ) <add_expr> }
<add_expr>   ::= <mul_expr> { ( "+" | "-" ) <mul_expr> }
<mul_expr>   ::= <unary_expr> { ( "*" | "/" ) <unary_expr> }
<unary_expr> ::= "-" <primary> | "!" <primary> | <primary>
<primary>   ::= INT_LIT | FLOAT_LIT | DURATION_LIT | WEIGHT_LIT
              | BOOL_LIT | STRING_LIT | IDENT | <func_call>
              | IDENT "." "setCount"
              | "(" <expr> ")"
```

Ambiguity note: both IDENT and IDENT "." "setCount" begin with an IDENT token, so a one-token lookahead is insufficient to distinguish them. The parser resolves this by consuming the IDENT and then peeking at the next token: if it is ".", it consumes it and expects "setCount"; otherwise the IDENT is treated as a variable reference or the start of a func_call. This is a standard two-token lookahead resolution and does not affect the unambiguity of the overall grammar.

```
<func_call> ::= IDENT "(" [ <arg_list> ] ")"
<arg_list>  ::= <expr> { "," <expr> }
```

Ambiguity Resolution Summary

Operator precedence: fully encoded in the seven grammar levels (unary > mul > add > relational > equality > logical-and > logical-or). No separate rule is needed; the grammar is unambiguous for expressions.

Dangling else: resolved by binding else to the nearest unmatched if — the standard greedy rule.

DURATION_LIT vs INT_LIT + IDENT: the lexer greedily consumes the numeric suffix (s, min, hr) as part of the token.

repeat vs while: repeat takes only an INT_LIT and the keyword weeks; while takes a general boolean expression. Critically, repeat provides no iteration counter variable to the body — the body cannot access which week it is currently executing. This is a deliberate restriction: fitness coaches should not be writing index arithmetic inside a training block. A counter variable inside repeat would invite the kind of off-by-one and index manipulation errors that the language is designed to prevent. This distinguishes repeat from a simple syntactic alias for while — it is a semantically restricted construct that eliminates a class of potential errors.

4.4 Semantics

WorkoutLang uses big-step operational semantics (natural semantics) as described in Sebesta §3.5. A judgement $\langle S, \sigma \rangle \Downarrow \sigma'$ reads: statement S , evaluated in state σ , terminates and produces state σ' . Expression evaluation is written $\langle e, \sigma \rangle \Downarrow v$. The state σ maps variable names to their current values.

Construct 1 — while Loop

Two rules are required because the loop has two distinct cases:

W-False: If the condition evaluates to false in the current state, the loop terminates immediately and σ is returned unchanged.

$$\frac{\langle e, \sigma \rangle \Downarrow \text{false}}{\langle \text{while } e \text{ B}, \sigma \rangle \Downarrow \sigma}$$

W-True: If the condition evaluates to true, the body B is executed producing σ' , and then the entire while statement is re-judged from σ' .

$$\frac{\langle e, \sigma \rangle \Downarrow \text{true} \quad \langle B, \sigma \rangle \Downarrow \sigma' \quad \langle \text{while } e \text{ B}, \sigma' \rangle \Downarrow \sigma''}{\langle \text{while } e \text{ B}, \sigma \rangle \Downarrow \sigma''}$$

Scope note: the rule abstracts over block-entry and block-exit scope management. In the rule, σ' is the state after B executes; per §4.6, any variables declared inside B are stack-dynamic and are destroyed on block exit, so σ' passed to the recursive judgement contains only the variables that existed before B was entered. Local variables declared inside B are not visible outside B.

Design decision revealed: the condition e is re-evaluated on every iteration against the updated state, not cached before the loop begins. Because WorkoutLang has no break statement, the only exit path is W-False — the loop must run until e becomes false.

Construct 2 — set Statement (Domain-Specific)

The set statement has exactly one rule, S-Set. The state σ is a pair (env, log) where $\sigma.\text{env}$ maps variable names to values and $\sigma.\text{log}$ is the accumulated session record (a sequence of set entries). The exercise name ex is a free-form identifier label — it is not a declared variable, so the premise checks that ex is a syntactically valid identifier ($\text{ex} \in \text{IDENT}$), not a membership check against $\sigma.\text{env}$. The other premises are range checks on the literal values.

$$\frac{\text{ex} \in \text{IDENT} \quad n \in \mathbb{Z}^+ \quad m \in \mathbb{Z}^+ \quad d \in \text{duration}}{\langle \text{set } \text{ex } \text{reps } n \times m \text{ rest } d, \sigma \rangle \Downarrow \sigma[\text{log} \mapsto \sigma.\text{log} ++ \{\text{ex}, n, m, d\}]}$$

Design decision revealed: because sets and reps are INT_LIT and rest is DURATION_LIT (enforced by the grammar in §4.3), their values are known at parse time — there is nothing to evaluate against state. The premises are therefore membership and positivity checks, not reduction steps. The append operator $++$ on $\sigma.\text{log}$ makes the domain-specific effect formally visible: a set statement does not produce a value, it records workout data into the log component of state. This is consistent with the statement-not-expression design in §4.7 and the reliability goal of §4.1.

Implementation note: in the interpreter, $\sigma.\text{log}$ is realised as the standard output channel rather than an in-memory sequence. Each application of S-Set writes exactly one entry to stdout, producing the same observable effect as an append to $\sigma.\text{log}$. This abstraction over the I/O channel is justified because WorkoutLang has no operation that reads back from the log — no program can distinguish an in-memory accumulation from a write-through to output. The formal rule remains correct; only the realisation of $\sigma.\text{log}$ differs.

4.5 Type System

Primitive Types and Value Sets

WorkoutLang has five primitive types. Each type has a fixed, distinct value set with no overlap, reflecting Sebesta §6.1 that a type is a collection of values and a set of operations defined on those values.

Type	Value Set	Operations
int	Signed 32-bit integers (-2,147,483,648 to 2,147,483,647)	+ - * / (integer division), == != < <= > >=
float	IEEE 754 double-precision floating-point values	+ - * / (real division), == != < <= > >=
bool	The two-element set {true, false}	&& !, == !=
duration	Non-negative integer paired with unit suffix (s, min, hr); stored internally as seconds	== != < <= > >=; no arithmetic
weight	Non-negative float paired with unit suffix (kg, lb); stored internally in kg	== != < <= > >=; no arithmetic

The unit-bearing types duration and weight are the primary domain-specific types. A duration is not an int with a suffix; it is a distinct type whose value set is meaningful only in a fitness context. Weight values are stored internally in kg regardless of input suffix: 135lb is stored as 61.235kg and printed in kg on output. This is a deliberate normalisation decision — the interpreter maintains a single canonical unit per physical quantity, consistent with the strong-typing principle that a weight is a weight regardless of how it was expressed.

Why no arithmetic on duration and weight? Sebesta §6.1 notes that the operations defined on a type are as important as the value set. Multiplying two weights (80kg * 60kg) produces a dimensionally nonsensical result. Restricting both types to comparison-only eliminates a family of type-correct but semantically wrong programs, consistent with the reliability priority declared in §4.1.

Note on float equality: the type table permits == and != on float values. IEEE 754 floating-point equality is semantically unreliable due to rounding (Sebesta §6.1, §7.3). WorkoutLang retains float equality because the language's fitness domain does not involve the kind of iterative floating-point computation where rounding accumulates. A coach comparing two weight values (80.0kg == 80.0kg) is comparing values entered directly, not computed. The design decision accepted is that the spec does not warn the user of this hazard — a future revision could restrict float equality to comparison with tolerance.

Strong Typing

WorkoutLang is both strongly typed (Sebesta §6.12) and statically type-checked (Sebesta §6.11). These are distinct properties. Strong typing (§6.12) means that every type error is always detected — it makes no claim about when. Static type checking (§6.11) means detection happens at compile time, before execution begins. WorkoutLang satisfies both: the type checker runs at compile time (static), and there are no operations that bypass type checking at any point (strong). For example: passing a duration where an int parameter is expected is caught at compile time; using a weight value as the condition of an if statement is caught at compile time; applying * to a bool operand is caught at compile time. No type error is deferred to runtime or silently ignored.

Implicit Type Coercion

WorkoutLang does not permit implicit type coercion (Sebesta §7.4). Every operand must already have the type expected by the operator or function signature. There are no widening conversions between types — not even from int to float. By requiring exact type matching, every type error surfaces as a clear message rather than a silent numeric conversion.

Type Equivalence Rule

WorkoutLang uses name equivalence (Sebesta §6.14) for all types. Two expressions are type-compatible if and only if they were declared with the same type name. There is no structural equivalence. This matters most for duration and weight: both are internally stored as numeric quantities, but they are not interchangeable. Under structural equivalence, a type checker might accept passing a duration where weight is expected. Name equivalence prevents it.

Structured Type: Day

WorkoutLang has one structured type: Day, a record (Sebesta §6.7) representing a named training day as declared inside a workout block. It exposes a single user-accessible field: `setCount` (type `int`), bound at compile time to the number of set statements in the block. The field is read-only; no assignment to `setCount` is permitted. The internal set list is not directly addressable by user code.

Name equivalence applies: two Day values are type-compatible only if both were declared with the day keyword. Day declarations have static lifetime (Sebesta §5.4.3) — they exist for the entire program duration, consistent with §4.6. No heap allocation is involved.

Field access uses dot notation: `LegDay.setCount`. Accessing `.setCount` on a non-Day identifier is a type error caught by the type checker.

Structured Type Design Decisions (Sebesta §6.5–6.7)

Field binding: field name (`setCount`) and type (`int`) are bound at compile time (§6.5). No dynamically added fields.

Field access: dot notation (§6.6), read-only from user code.

Type equivalence for Day: name equivalence, consistent with §4.5.

Memory: Day declarations have static lifetime (§5.4.3). Stack-dynamic allocation is not used for Day values.

4.6 Names, Binding, Scope, and Lifetime

Legal Identifiers

An identifier must match `[a-zA-Z_][a-zA-Z0-9_]*`. Identifiers are case-sensitive. All keywords and type names are reserved and may not be used as identifiers. The token `x` is also reserved as a separator in set statements.

Bindings: Compile Time vs. Run Time

What	When	Sebesta
workout / routine / func names	Compile time — fixed before execution	§5.3
Type and arity of declarations	Compile time	§5.3
Function parameter values	Run time — on call	§5.4.2
Local variable values	Run time — on block entry	§5.4.2

Scoping Rule: Static (Lexical) Scoping

WorkoutLang uses static scoping (Sebesta §5.4). The scope of every name is determined by the textual structure of the program, not by the call sequence at runtime. There are three scope levels:

Global scope: workout, routine, and function declarations. Visible throughout the entire program.

Routine/function scope: parameters and local variables declared inside a routine or function body. Visible only within that block.

Block scope: variables declared inside an if, while, or repeat block. Visible only within that block and any nested blocks.

Why not dynamic scoping? Under dynamic scoping, a function call could accidentally capture variables from the calling routine. This would make programs extremely hard to read — violating the primary design goal stated in §4.1. Static scoping makes every name resolution traceable to its textual declaration, which is essential for a language used by non-programmers.

Lifetime of Variables

Variable Kind	Lifetime	Sebesta Reference
workout / routine / func names	Static — exist for entire program duration	§5.4.3
Function parameters	Stack-dynamic — created on call, destroyed on return	§5.4.2
Local variables (routine/func)	Stack-dynamic — created on block entry, destroyed on exit	§5.4.2
Block variables (if/while/repeat)	Stack-dynamic — created on block entry, destroyed on exit	§5.4.2

Assignment scope rule: `<assign_stmt> ::= IDENT "=" <expr>` permits assignment to any identifier syntactically, but the type checker enforces that the left-hand side must resolve to a mutable variable in the current scope — not a workout, routine, function, or Day declaration. Assigning to a non-variable name (e.g. `UpperBody = 5` where `UpperBody` is a workout declaration) is a type error caught at compile time.

Parameter passing mode: parameters are passed by value (Sebesta §9.2). The called function receives a copy of the argument; modifications to a parameter inside the function body do not affect the caller's variables. This is consistent with the stack-dynamic lifetime

model of §4.6 and with the readability goal of §4.1 — call-by-reference would allow functions to silently modify the caller's state, which is invisible to a non-programmer reading the function in isolation.

WorkoutLang does not support explicit heap allocation. There is no `malloc`, `new`, or pointer type. This is a deliberate design decision: the target users are fitness coaches, not programmers. Heap management would add cognitive overhead without any benefit in this domain.

4.7 Expressions and Assignment

Operator Precedence and Associativity (Sebesta §7.2, highest to lowest)

Level	Operators	Associativity
1 — Unary	- !	Right
2 — Multiplicative	* /	Left
3 — Additive	+ -	Left
4 — Relational	< <= > >=	Left
5 — Equality	== !=	Left
6 — Logical AND	&&	Left
7 — Logical OR		Left (lowest precedence)

Precedence is encoded directly in the seven-level grammar hierarchy in §4.3. No separate rule is needed; the grammar is unambiguous for expressions. `A || B && C` parses as `A || (B && C)`, consistent with standard boolean algebra.

Short-Circuit Evaluation (Sebesta §7.7)

`&&` and `||` use short-circuit (lazy) evaluation. For `A && B`, `B` is not evaluated if `A` is false. For `A || B`, `B` is not evaluated if `A` is true. This is a reliability decision: a fitness coach may write a compound condition without worrying whether the right operand is safe to evaluate independently.

Order of Operand Evaluation (Sebesta §7.6)

Left-to-right, fully defined. WorkoutLang guarantees evaluation order for all binary operators. This eliminates the class of side-effect-dependent bugs Sebesta describes in §7.6 for languages like C where evaluation order is unspecified. The accepted trade-off is that the compiler cannot reorder subexpressions for optimization.

Assignment (Sebesta §7.7.1)

Assignment is a statement, not an expression. The form `IDENT = <expr>` produces no value and cannot appear inside a larger expression. This prevents the classic `if (x = 5)` reliability hazard Sebesta identifies in §7.7.1. The grammar enforces this: `<assign_stmt>` appears only in `<stmt>`, never in `<expr>`.

Mixed-Mode Arithmetic (Sebesta §7.4)

Not permitted. WorkoutLang has no implicit coercion (§4.5), so `int + float` is a type error caught by the type checker. Every operand of a binary operator must have the same type. duration and weight operands are additionally restricted from arithmetic entirely (§4.5).

4.8 Design Rationale

1. Strong typing with no implicit coercion (§4.5)

I chose strong static typing with zero implicit coercion because WorkoutLang's users are fitness coaches, not programmers — they will not debug type errors that silently produce wrong numbers. Sebesta §6.12 identifies strong typing as the property that eliminates entire classes of type errors by guaranteeing every operand is checked before use; I apply this maximally by rejecting even the `int-to-float` widening that Sebesta §7.4 calls the least dangerous coercion. The practical justification is domain-specific: a coach who writes a volume calculation and accidentally mixes a repetition count with a rest duration should get an error message, not a silently computed nonsense figure that ends up in a training plan. The trade-off I accepted is that users must write an explicit conversion function whenever they need to combine numeric values of different types — a minor inconvenience for a programmer, but an acceptable one for a tool whose programs are short and whose users write them deliberately.

2. Name equivalence for all types including Day (§4.5)

I chose name equivalence (Sebesta §6.14) over structural equivalence for every type in WorkoutLang, including the `Day` record. The most concrete justification comes from types that already exist in the language: duration and weight are both internally stored as numeric quantities (seconds and kilograms respectively). Under structural equivalence, a type checker inspecting only the internal representation could treat them as interchangeable — a duration value could be passed where a weight is expected, because structurally both are just numbers. Name equivalence prevents this: duration is duration because it was declared with the duration keyword, and weight is weight because it was declared with weight. No structural similarity can override the declared name. For fitness coaches, this distinction is essential — a rest period of 90s and a load of 80kg are categorically different things even though both reduce to a number internally. The trade-off is rigidity: users cannot pass a duration where a weight is expected even if the numeric value would make sense in isolation. I accept that cost because the prevented errors — silently treating rest periods as weights in a training calculation — are more costly than the inconvenience of explicit type declarations.

3. No arithmetic on duration and weight (§4.5)

I restricted duration and weight to comparison operators only, with no arithmetic, because the operations that are meaningful in this domain are not the same as the operations that are type-correct in a purely numeric sense. Sebesta §6.1 establishes that a type is defined by both its value set and the operations permitted on it; I use this to draw the boundary deliberately. Adding two rest periods (90s + 60s) is a plausible operation, but multiplying two weights (80kg * 60kg) produces a dimensionally absurd result — `kg^2` has no meaning in a training program. Rather than permit some arithmetic and prohibit other arithmetic through runtime checks, I prohibit it entirely at the type level, which keeps the type system simple and the error messages clear. The trade-off is that a coach who genuinely wants to compute total rest time must extract the numeric component through a function.

4. Assignment as statement, not expression (§4.7)

I made assignment a statement in WorkoutLang because the language's target users have no background in C-family languages and no reason to expect that `=` can appear inside a condition. Sebesta §7.7.1 identifies the assignment-as-expression design as a known source of reliability hazards, citing the `if (x = 5)` class of bugs that arise precisely because the programmer conflates assignment and equality testing. For a fitness coach writing a routine, the distinction between `=` and `==` is already a cognitive load; allowing `=` in expression position would make that load worse without providing any benefit. The trade-off is that certain compact idioms available in C or Python are not expressible in WorkoutLang — I accept this because the language is narrow by design.

5. Static scoping with no heap allocation (§4.6)

I chose static (lexical) scoping and deliberately excluded heap allocation because both decisions serve the same goal: making every name resolution in a WorkoutLang program traceable to a specific textual declaration without any runtime bookkeeping visible to the user. Sebesta §5.4 establishes that static scoping ties name resolution to the program's textual structure; this means a coach reading a routine can identify where every variable comes from by looking at the enclosing blocks, without reasoning about what called what at runtime. Dynamic scoping would allow a called function to silently capture a variable from the calling routine — invisible to a non-programmer reading the function in isolation. Heap allocation is excluded for the same readability reason: explicit memory management would require the user to reason about object lifetimes, a concept with no analogue in workout programming. The trade-off is that WorkoutLang cannot represent recursive data structures or dynamically sized collections — I accept this because the fitness domain does not require them.

6. repeat provides no iteration counter (§4.3)

I designed `repeat N weeks` to provide no accessible iteration counter variable to its body. This is a deliberate reliability decision (Sebesta §1.3.3): a fitness coach writing a periodization block should not be able to write index arithmetic inside the loop body. If a counter were available, users could write conditions like `if week == 3 { load HeavyDay }` inside the `repeat` body — introducing the kind of off-by-one and index manipulation errors that a DSL for non-programmers should eliminate by design. The `while` loop already serves users who need a general counter-driven loop; `repeat` is intentionally more restricted. The trade-off is expressiveness: a coach who genuinely needs week-specific logic inside a loop must use `while` with an explicit counter variable. I accept this because the use case (a fixed block of identical training weeks) is the overwhelmingly common one in fitness programming, and the restriction enforces that pattern.